

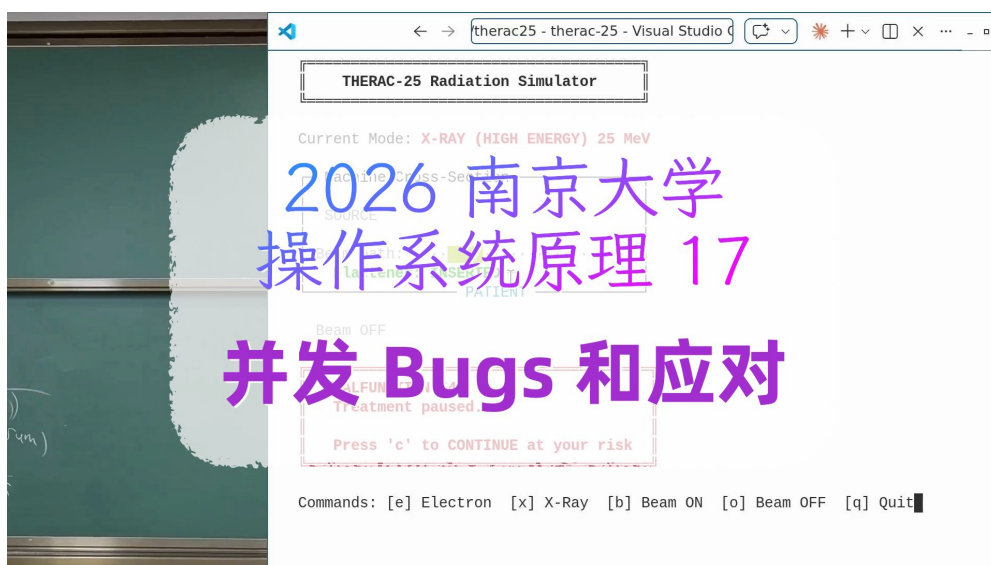
课程笔记

并发 Bugs 与应对

南京大学 操作系统原理 2026 春季学期 第 17 讲

lecture-to-notes

2026 年 5 月 7 日



视频作者/频道：绿导师原谅你了（蒋炎岩 jyy）

发布日期：2026 年春季学期

视频时长：1 小时 28 分钟

视频链接：<https://www.bilibili.com/video/BV1Jg96BjE68>

目录

1 引言：并发为什么这么难	2
2 死锁与四个必要条件	2
2.1 死锁的定义与现实例子	2
2.2 AA 型死锁：自己等自己	3
2.3 ABBA 型死锁：环路等待	4
2.4 死锁的四个必要条件（1971 经典论文）	4
2.5 本章小结	5
3 在代码中避免死锁：lockdep & LD_PRELOAD	6
3.1 动态检测：构建上锁顺序图	6
3.2 LD_PRELOAD：拦截 mutex API	7
3.3 本章小结	8
4 数据竞争（Data Race）	8
4.1 什么是 race condition?	8
4.2 ThreadSanitizer：动态检测 race	9
4.3 Therac-25：要人命的并发 bug	10
4.4 本章小结	10
5 更隐蔽的并发 Bugs	10
5.1 原子性违反（Atomicity Violation）	10
5.2 ABA 问题	11
5.3 TOCTOU：Time of Check to Time of Use	12
5.4 顺序违反（Order Violation）	12
5.5 本章小结	13
6 总结与延伸	13
6.1 讲师的核心论断	13
6.2 我的结构化提炼	13
6.3 开放问题与扩展方向	14
6.4 拓展阅读	14

1 引言：并发为什么这么难

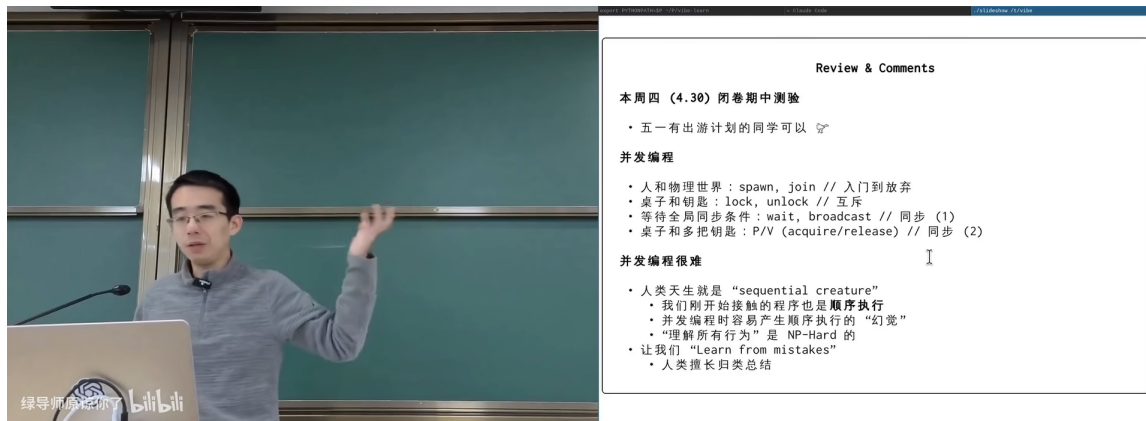


图 1: 并发编程的全貌：Spawn / Join / Lock / Unlock / Wait / Broadcast / P / V——讲师说”理论上你已经能写所有并发程序了”²

到此为止，并发编程在**原语层面**已经讲完。Spawn / Join / Lock / Unlock / Wait / Broadcast / P / V，理论上配合系统调用 clone 与 futex，能实现任何并发程序。但讲师反复强调：

人类的物理直觉对”异步”是天生不友好的

讲师的核心论断：

”我们人类整个训练过程都在物理世界中完成。脑子在脑壳里飞不出来，眼睛能看到的就那么大。所以人天生就是在’共享内存’里视角非常小的一个东西。”

后果：从顺序程序变到并发程序，会自然地产生”它应该顺序执行”的幻觉，错就错在这。要真正理解一个并发程序的行为，必须**枚举指数多个交错**才能确定它**真正的行为**——这就是 deep PSPACE-hard。

所以本节课要做的是 **Learning from Mistakes**：

1. 死锁 (Deadlock)
2. 数据竞争 (Data Race)
3. 原子性违反 / 顺序违反 / ABA / TOCTOU 等更隐蔽的并发 bug

2 死锁与四个必要条件

2.1 死锁的定义与现实例子

Deadlock 的形式定义：a state in which each member of a group is waiting for another member, including itself, to take action. 字面意思：每个线程都在等其他线程动一下，但谁都不动。

²视频画面时间区间：00:00:30-00:01:00。

现实中的死锁

讲师举了一个生动的例子：去年 11 月一位同学投稿的真实路口照片——**多辆公交车头尾、头尾相接，没有一辆能再向前移动一步**。物理上、逻辑上都进入了 deadlock。条件其实很苛刻，但一旦满足就互相锁死。

2.2 AA 型死锁：自己等自己

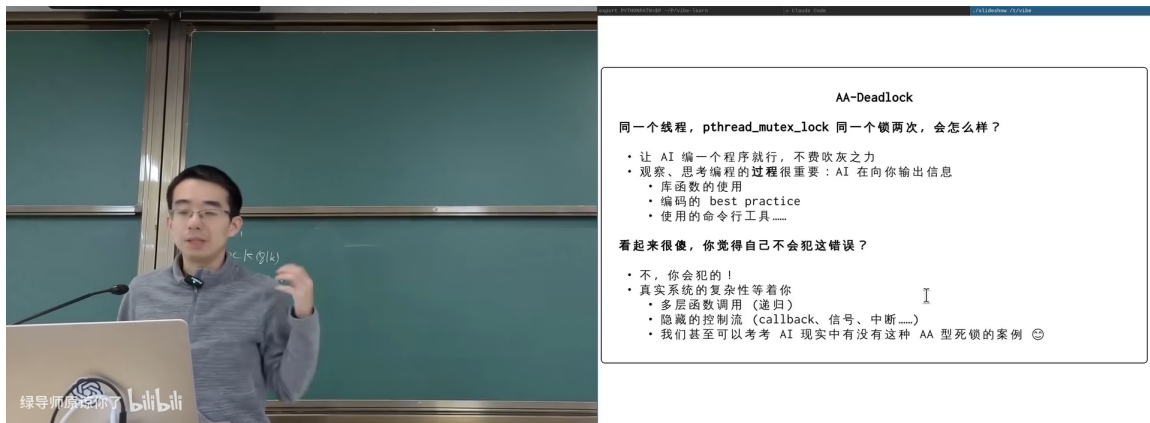


图 2: AA 型死锁：同一线程对同一把非递归锁连续 lock 两次，自己等自己³

```
1 mutex_t lock;
2
3 int main() {
4     mutex_init(&lock);
5     mutex_lock(&lock);
6     printf("first_lock_acquired\n");
7     mutex_lock(&lock);          /* 同一线程再次 lock —— 卡死 */
8     printf("second_lock_acquired\n"); /* 永远不会打印 */
9 }
```

Listing 1: 最简 AA 死锁

为什么写代码时还是会犯？

看这个 demo 你会觉得”绝对不会写这种代码”。但真实系统里：

- 代码散落在几十个文件、多层函数调用、甚至递归里
- 信号、中断、callback 隐藏了控制流——你 acquire 了一把锁，然后 callback 触发，callback 内部认为它要保护数据结构也 acquire 同一把锁
- POSIX pthread_mutex 默认是非递归的：第二次 lock 会阻塞自己

所以 AA 型死锁不傻——它在生产环境里依然时不时地发生。

³视频画面时间区间：00:05:00-00:06:30。AI vibe code 一个 demo 就能复现。

2.3 ABBA 型死锁：环路等待

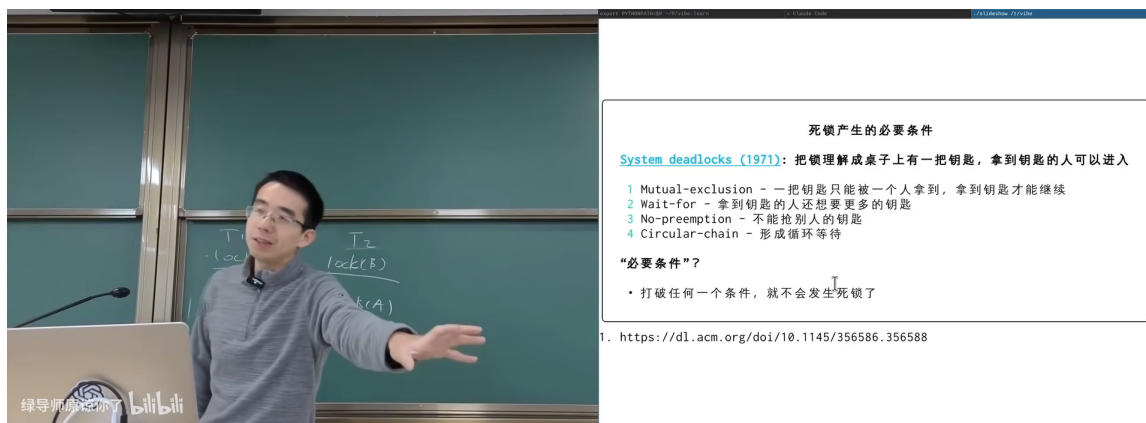


图 3: ABBA 死锁: T1 持 A 等 B, T2 持 B 等 A, 相互堵死⁴

```
1 /* Thread T1 */           /* Thread T2 */
2 mutex_lock(&A);           mutex_lock(&B);
3 mutex_lock(&B); /* wait! */ mutex_lock(&A); /* wait! */
```

Listing 2: ABBA 死锁的最简形态

Dining Philosophers 是 ABBA 的环形推广: 5 位哲学家同时拿起左手叉子, 每个人都在等右邻居放下叉子, 形成环。

好消息: 死锁的 **symptom** 比较容易检测——程序原本应该在毫秒级输出, 突然全静默了; 用 GDB attach 上去能看到所有线程都阻塞在 `futex_wait`。

2.4 死锁的四个必要条件 (1971 经典论文)

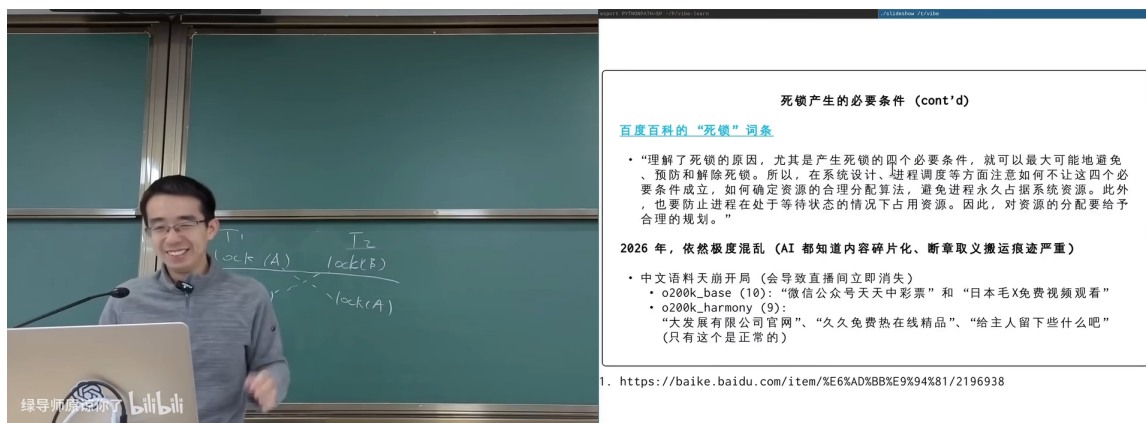


图 4: System Deadlocks (Coffman et al., 1971) 总结的四条必要条件⁵

⁴视频画面时间区间: 00:10:00–00:11:30。dining philosophers 是其推广。

⁵视频画面时间区间: 00:13:30–00:15:30。讲师推荐的经典论文。

死锁四要素（一个不满足就不会死锁）

1. **Mutual Exclusion**（互斥占用）：资源排他，任意时刻最多一人持有
2. **Wait-For**（持有并等待）：拿到一把还想要更多
3. **No Preemption**（不可抢占）：谁也抢不走别人的钥匙
4. **Circular Wait**（循环等待）：等待图里存在环

所谓必要条件 = 任意打破一条，死锁就不会发生。所以解决死锁的所有方法都是打破其中一个或多个条件。

一一对应到 AA 与 ABBA：

死锁形态	关键条件	表现
AA	wait-for（自己拿了还想拿）	拿了 A 还想再要一把 A
ABBA	circular wait（环路等待）	T1 持 A 等 B、T2 持 B 等 A 形成环
Dining philosophers	circular wait（5 节点环）	五位哲学家都拿左手

百度百科的”死锁四要素”是错的

讲师每年都会拿出来鞭策。坊间流传的版本常常忘了”所谓必要条件 = 打破任意一个就不会发生”这层语义，把它说成”同时满足这四个就会死锁”——完全反了。这种错误广泛存在于网上速记题型里，要小心。

2.5 本章小结

- AA / ABBA / Dining Philosophers 都是死锁的不同形态
- 1971 年的经典论文给出了四条必要条件
- 任意打破一个条件 → 死锁无法发生
- 死锁的好处：症状明显，容易被检测

3 在代码中避免死锁：lockdep & LD_PRELOAD

3.1 动态检测：构建上锁顺序图

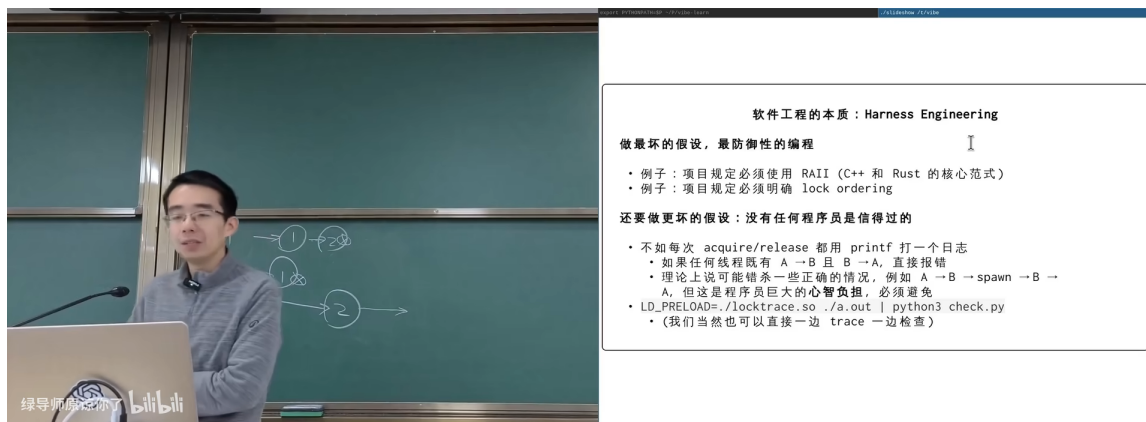


图 5: lockdep 思路：把所有线程上锁顺序记下来，构建有向图，检测有没有环⁶

核心想法：把上锁顺序变成图问题

对系统中所有 lock(X) 操作做 trace：

- 当线程 T 持有锁集合 {A} 时再次 lock(B)，记下有向边 A → B (“在持 A 的情况下又请求 B”)
- 所有线程的上锁序列合在一起得到一张全局上锁顺序图
- 检测图里有没有环 → 有环说明存在 ABBA 风险

哪怕某次具体执行没有真死锁（例如调度刚好错开），只要图里有环，就潜在有 ABBA bug——这种代码必须改写。讲师反复强调：“不要只防止 bug 真的爆发，要防止潜在的可能。”

⁶ 视频画面时间区间：00:38:00–00:40:00。

3.2 LD_PRELOAD: 拦截 mutex API

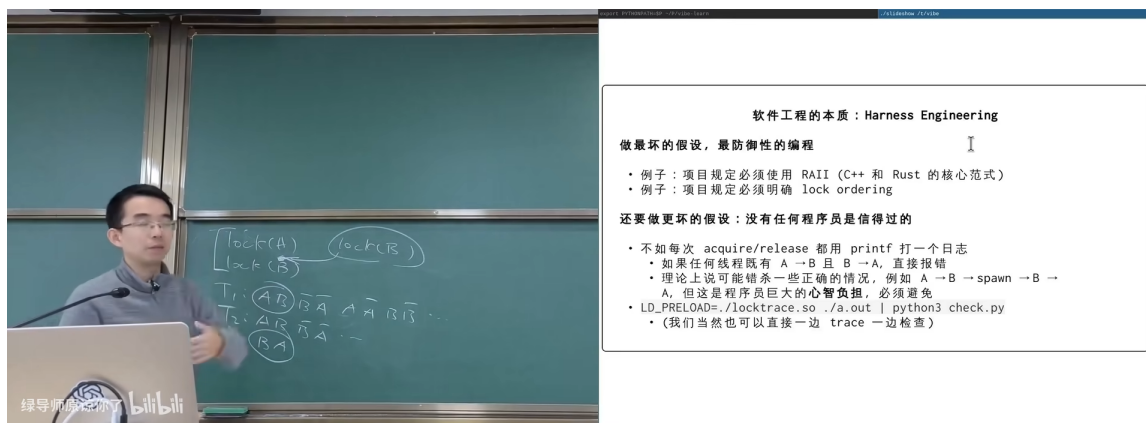


图 6: 用 LD_PRELOAD 在 pthread_mutex_lock/unlock 处插桩——程序无需重编译⁷

机制：

1. 写一份共享库 libdeadlockcheck.so, 里面定义了同名函数 pthread_mutex_lock / pthread_mutex_unlock
2. 函数内部：先调用真正的 libc 实现，然后记录 trace
3. 用 LD_PRELOAD=./libdeadlockcheck.so ./your_program 启动
4. 动态链接器会用我们的实现覆盖 libc 里的同名符号

讲师演示：用 AI vibe-code 一个最简版本只要几分钟——核心代码就是”维护一个 thread_local 的锁栈 + 全局图”。

```
1 __thread Stack held_locks;          /* 当前线程持有的锁 */
2 Graph global_order;                /* 全局上锁顺序图 */
3
4 int pthread_mutex_lock(mutex_t *m) {
5     int rc = real_pthread_mutex_lock(m);
6     if (rc == 0) {
7         for (auto h : held_locks)
8             global_order.add_edge(h, m);
9         held_locks.push(m);
10        if (global_order.has_cycle())
11            warn("potential ABBA deadlock!");
12    }
13    return rc;
14 }
15 int pthread_mutex_unlock(mutex_t *m) {
16     held_locks.remove(m);
17     return real_pthread_mutex_unlock(m);
18 }
```

Listing 3: lockdep 核心数据结构（伪代码）

⁷视频画面时间区间：00:40:00–00:41:30。

Linux 内核的 lockdep

内核版的 lockdep 同思想，但更复杂：

- 处理 spinlock / rwlock / rcu 等多种锁类型
- 区分中断上下文、抢占上下文
- 在内核启动早期就开始追踪

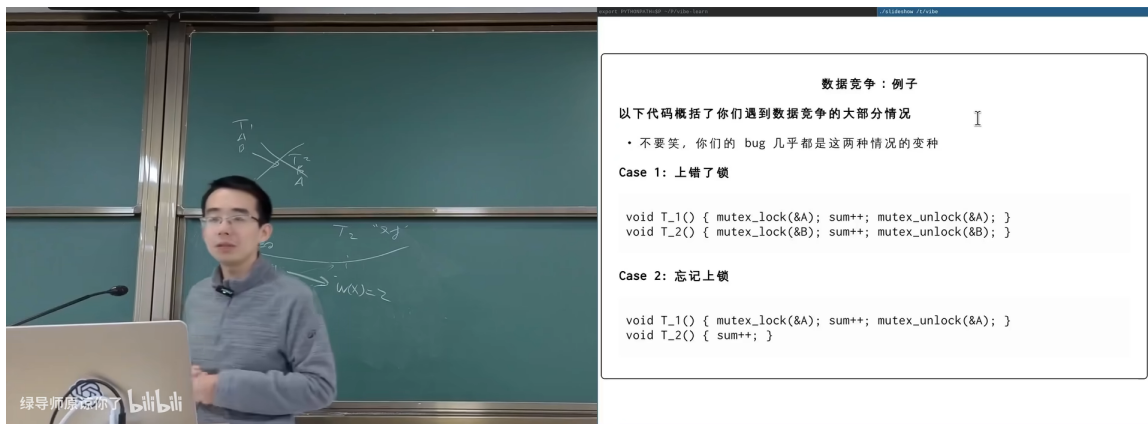
开启后内核稍慢，但能预先发现大量隐藏的锁顺序问题——Linux 内核里相当一部分锁相关 bug 是 lockdep 找出来的。

3.3 本章小结

- 死锁检测可以归约为图算法：构建上锁顺序图、查环
- LD_PRELOAD 让我们无需重编译就能给任何程序加 trace
- AI 时代写一个能用的 lockdep 只是几分钟的事——Systems 程序员的家底被抄了

4 数据竞争 (Data Race)

4.1 什么是 race condition ?



数据竞争：例子

以下代码概括了你们遇到数据竞争的大部分情况

- 不要笑，你们的 bug 几乎都是这两种情况的变种

Case 1: 上错了锁

```
void T_1() { mutex_lock(&A); sum++; mutex_unlock(&A); }
void T_2() { mutex_lock(&B); sum++; mutex_unlock(&B); }
```

Case 2: 忘记上锁

```
void T_1() { mutex_lock(&A); sum++; mutex_unlock(&A); }
void T_2() { sum++; }
```

图 7: Race condition: 两个线程在没有同步的情况下并发访问同一变量，且至少一个是写⁸

Race condition 的三要素

1. 至少两个线程并发地访问同一内存位置
2. 至少有一个是写
3. 它们之间没有 happens-before 关系（没有 mutex / cv / sem 等同步原语保护）

只要这三条同时成立，行为就是 **undefined behavior**（注意这里是并发的 UB，不完全等同于 C 语言的 UB；C 语言的 UB 包括各种诡异的可能；并发 UB 主要是结果不可预测）。

⁸视频画面时间区间：00:48:00–00:50:00。

4.2 ThreadSanitizer: 动态检测 race

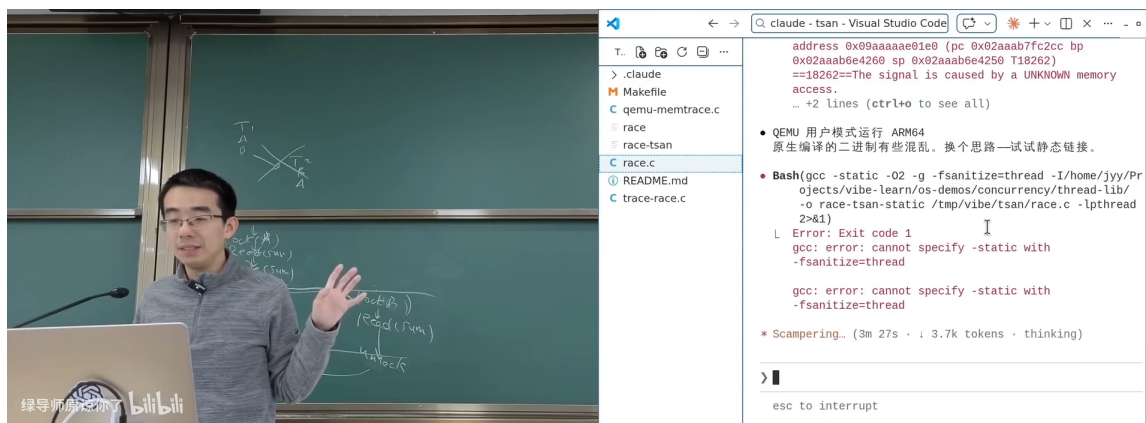


图 8: ThreadSanitizer (TSan): 编译期插桩 + 运行期记录访问历史, 用 happens-before 推理检测竞争¹⁰

```
1 gcc -fsanitize=thread -g your_program.c -o your_program
2 ./your_program # TSan 在运行时检测并打印 race
```

Listing 4: 用 TSan 编译

TSan 工作原理:

1. 编译期对每条内存访问插桩
2. 维护每个内存位置的“vector clock”: 最后是哪个线程在哪个时间点访问的
3. 检查每对访问之间是否存在 happens-before 关系 (mutex / cv / atomic 等可建立)
4. 没有 happens-before 且至少一个是写 → 报告 race

lockdep / ThreadSanitizer / Helgrind / Eraser

- **Eraser** (1997): 第一个 dynamic race detector, 提出 *lockset algorithm*
- **Helgrind** (Valgrind 套件): 基于 Eraser 思想, 无需重编译
- **ThreadSanitizer** (Google 2009): 编译插桩, 性能远好于 Valgrind
- **Linux kernel KCSAN** (2019): 内核级 race detector

讲师说: “这些工具曾经是 systems programmer 的看家本领。今天 AI vibe code 一下就能搓个像样的版本——家底都被抄了。”

¹⁰ 视频画面时间区间: 00:55:00–00:59:00。讲师 vibe-code 编译 QEMU 时把笔记本编译过热死机了。

4.3 Therac-25: 要人命的并发 bug

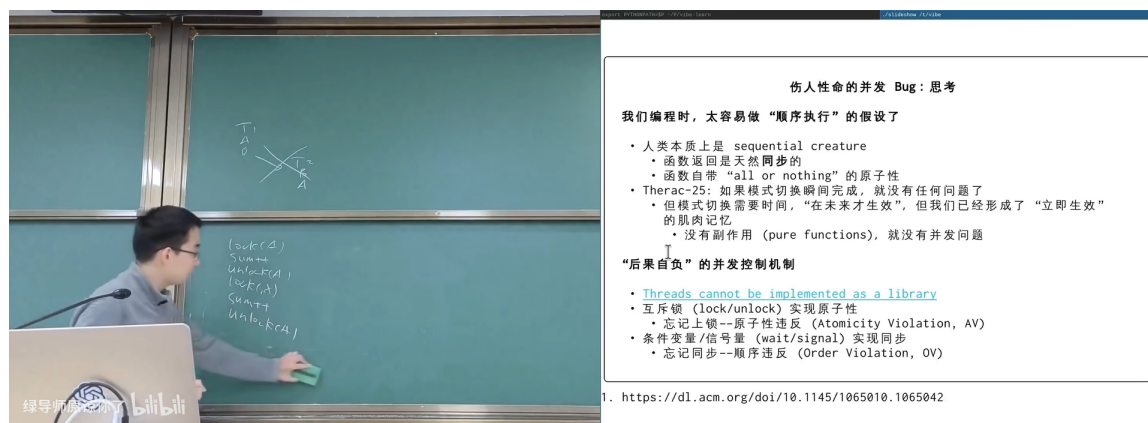


图 9: Therac-25 放射治疗仪 1985-1987 因并发 bug 导致至少 6 人死亡的真实事故¹¹

真实的、要命的并发 bug

Therac-25 是一台放射线治疗仪，能精确地把高能 X 射线聚焦到病灶。它有两种模式：

- 低剂量电子束（搭配偏转磁铁聚焦）
- 高剂量 X 射线（搭配钨靶板，磁铁也要在位）

Bug: 操作员在终端切换模式时，如果 8 秒内做完一系列操作，软件来不及更新硬件状态——病人会被高剂量 X 射线无遮挡地直接照射，没有钨靶板衰减。

至少 6 人受到致命剂量。其根因是事件驱动的 **race condition**: UI 线程 `set_mode(electron)` 与硬件初始化线程 `move_target_in()` 之间没有正确同步。

讲师用这个例子提醒：并发 bug 不是程序员的玩笑，是真正会要命的事。

4.4 本章小结

- Race = 并发访问 + 至少一写 + 无同步
- 静态分析 + 动态插桩是发现 race 的两条主路
- ThreadSanitizer 是工业级最常用工具
- Therac-25 提醒：写并发的人对生命安全负有真实责任

5 更隐蔽的并发 Bugs

5.1 原子性违反 (Atomicity Violation)

Atomicity violation = 程序员以为一段代码是原子的，但实际上它能被打断。最典型的形态是 **check-then-act**:

¹¹ 视频画面时间区间：01:02:30–01:05:00。

```

1 if (ptr != NULL) {
2     /* —— 这里被另一线程打断, ptr 可能被改成 NULL —— */
3     *ptr = 42;          /* CRASH */
4 }

```

Listing 5: 经典的原子性违反

check-then-act 必须用锁包起来

不是检查不对，是检查和使用之间没有锁保护——条件变量解决的就是这个问题：

```

1 mutex_lock(&m);
2 while (ptr == NULL) cond_wait(&cv, &m);
3 *ptr = 42;          /* 这里 ptr 一定不是 NULL, 且 m 在手 */
4 mutex_unlock(&m);

```

Listing 6: 条件变量保证 check-then-act 原子

当你在 while + cond_wait 模板里跑出来时，条件被验证为真且锁在手——*ptr = 42 真的安全。

5.2 ABA 问题

ABA 问题的形态：你看到一个变量从值 A 经过一段时间后又是 A，于是认为“它没变过”——但其实它先变成了 B 然后又变回了 A。在 lock-free 数据结构和 CAS 操作里特别常见。

Use-After-Free 也是 ABA 的一种

- 线程 1：拿到指针 p 指向对象 X
- 线程 2：free(X)，分配器把这块内存还给堆
- 线程 2：在同地址再 malloc 一个对象 Y
- 线程 1：以为还指着 X，写入数据 → 实际上写到了 Y

内核里大量的 Use-After-Free 安全漏洞本质都是 ABA。

5.3 TOCTOU: Time of Check to Time of Use

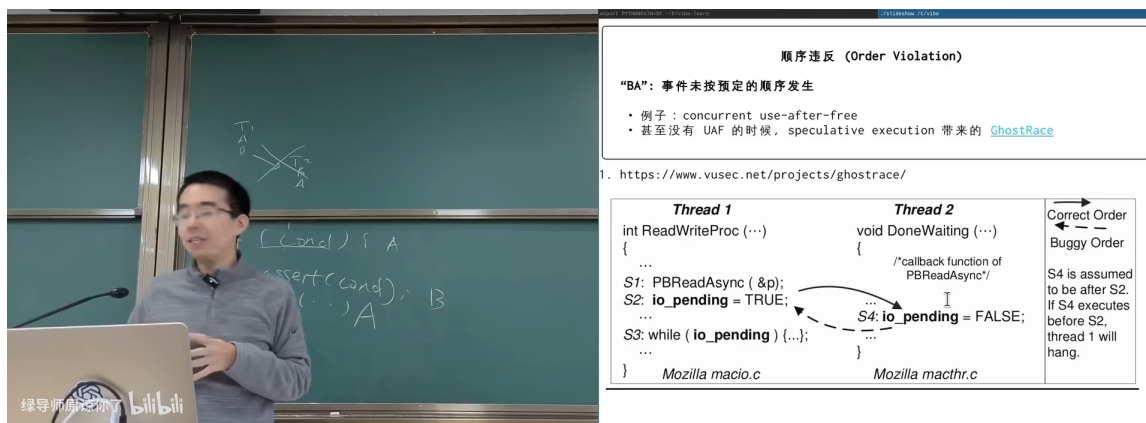


图 10: TOCTOU: 检查时间与使用时间之间存在时间差，攻击者可以乘机修改¹²

TOCTOU 的经典 sendmail 攻击

1980 年代 Unix 上的 sendmail 是 setuid root 程序：

1. 它先 `lstat(/var/mail/alice)` 检查这个 mailbox 不是符号链接
2. 在 `check` 与 `open` 之间，攻击者把 `/var/mail/alice` 替换成符号链接，指向 `/etc/passwd`
3. sendmail 以 root 权限 `open(/var/mail/alice, O_APPEND)` 写入”邮件”
4. 内核解析符号链接，实际写到了 `/etc/passwd`！

如果攻击者构造的”邮件正文”是合法的 `/etc/passwd` 行（含 `root::0:0:... 这种`），就添加了一个无密码的 `root` 账号——系统直接被攻破。

修复：使用 `open(... O_NOFOLLOW)`，或先 `open` 拿 `fd` 再 `fstat` 检查文件类型——在持有 `fd` 的时候 `inode` 不会变。

5.4 顺序违反 (Order Violation)

Order violation = 程序员假定 A 一定先于 B，但没用同步保证：

```
1 /* Thread 1: 初始化 */          /* Thread 2: 使用 */
2 config = load_config();          /* 没有等 Thread 1 完成 */
3 ready = true;                    while (!ready) /* spin */;
4                                 use(config);      /* 可能用到部分初始化的 config */
```

Listing 7: 典型的 order violation

Bug: `config = ...` 与 `ready = true` 之间没有 memory barrier，编译器或 CPU 可能重排两条赋值，让 `ready` 先变 `true`。修复：用 `atomic + release/acquire`，或直接条件变量。

¹² 视频画面时间区间：01:18:00–01:21:00。

5.5 本章小结

- Atomicity violation: check-then-act 必须用锁包
- ABA / Use-After-Free: 相同值不等于相同含义
- TOCTOU: 检查与使用之间有窗口, 会被攻击
- Order violation: 编译器和 CPU 都可能重排, 需要 atomic / barrier

6 总结与延伸

6.1 讲师的核心论断

并发 bug 的”统一视角”

所有并发 bug 都来自程序员对执行顺序的假定与真实并发执行之间的鸿沟:

- 死锁: 以为”我能拿到所有锁”——结果环路等待
- Race: 以为”我读到的值不会变”——结果别人在写
- Atomicity violation: 以为”check-then-act 是原子的”——结果不是
- ABA: 以为”值没变就是状态没变”——结果中间转了一圈
- TOCTOU: 以为”检查后立即使用是安全的”——结果中间被换了
- Order violation: 以为”代码顺序就是执行顺序”——结果被重排

治理思路一致: 用同步原语 (mutex / cv / sem / atomic) 建立 happens-before, 让程序员的假定真正成立。

6.2 我的结构化提炼

	Bug 类型	关键症状	检测工具	防御
并发 bug 与对应工具/对策	Deadlock	全静默卡死	lockdep, GDB attach	破坏、统一锁
	Data Race	偶发错误结果	ThreadSanitizer, Helgrind	mutex / atom
	Atomicity	check-then-act 失效	TSan + assert	用 cv + while
	ABA / UAF	用到错值/已释放	ASan, KCSAN	引用计数、版
	TOCTOU	安全攻击窗口	静态审计	O_NOFOLLOW, f
	Order Violation	init 未完先 use	TSan	atomic release

两条心法

1. 所有并发 bug 都假设了某种”顺序”——把”我假设的顺序”显式写成同步原语, bug 就消失了

2. **Tools first, intuition second**——TSan / lockdep / GDB 是你最好的朋友，比”想清楚”靠谱得多

6.3 开放问题与扩展方向

- **Lock-free 数据结构**用 CAS 避免锁，但 ABA 风险更高——业界用 hazard pointer / epoch reclamation
- **Memory model** (C++11、Java、x86 TSO 等) 规定了”哪些重排是允许的”，是写正确并发代码的另一个核心知识
- **Formal verification**: TLA+、Spin、Coq 能数学证明并发协议的正确性
- **AI 时代的并发**: LLM agent 编排本身就是并发系统；下游 RPC、retry、超时都会带来新的并发 bug 形态

6.4 拓展阅读

- Coffman, Elphick, Shoshani, "System Deadlocks," *Computing Surveys*, 1971
- Lu et al., "Learning from Mistakes: A Comprehensive Study on Real-World Concurrency Bug Characteristics," *ASPLOS* 2008 (本章名字的来源)
- Savage et al., "Eraser: A Dynamic Data Race Detector for Multithreaded Programs," *TOCS* 1997
- Leveson, "An Investigation of the Therac-25 Accidents," *IEEE Computer*, 1993
- OSTEP 第 32 章 Common Concurrency Problems